Sanjivani Rural Education Society's

Sanjivani College of Engineering, Kopargaon-423 603

Project Title

SafePilot: Real-Time ADAS Perception Dashboard

An AI-Powered Driver-Assistance System with an Interactive Web Dashboard

Presented by

Sr.	Name	Roll No.
1	Uday Annasaheb Thete	0124ucsm2005
2	Parth Milind Kulkarni	0124ucsm2003
3	Mahesh Tulsiram Shinde	0124ucsm2008
4	Vaibhav Nitin Khandekar	0124ucsm2007
5	Krushna Prashant Warule	0124ucsm2011

Project Guide: Prof. S.A. Shivarkar

Subject: Advance Data Science

Department of Computer Engineering

Background / Problem Statement

What problem does the project address?

- Delayed driver reaction: late braking, unnoticed vehicles, lane drift
- Commercial ADAS relies on expensive radar and LiDAR hardware
- A low-cost system using only a single camera is needed
- The original version ran only from a command line, hard to demo, monitor or control

Why is it important?

- Road accidents are often caused by delayed reaction
- Expensive sensors limit access to driver-assistance research
- A dashboard makes results easy to demonstrate, monitor and control

Proposed Solution

SafePilot analyses dashcam video in real time

- Detects and tracks vehicles and pedestrians
- Estimates the distance of each object
- Predicts collision risk using time-to-collision (TTC)
- Monitors lane position and lane departure
- Shows everything on a live web dashboard

Single low-cost camera

Open-source AI models, no radar or LiDAR

Interactive dashboard

Click any object to lock on; thresholds adjustable live

Improved over the command-line version

Browser dashboard, aspect ratio preserved, TTC in real seconds

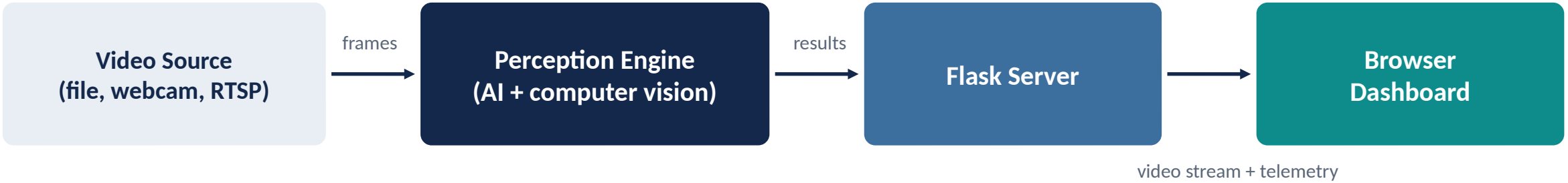
Objectives

1. Detect and track road users in real time
2. Estimate distance and time-to-collision (TTC) from a single camera
3. Classify collision risk as Safe, Caution, Warning or Critical
4. Detect lane position and lane departure
5. Present all results in an interactive browser-based dashboard

Scope

- Input: video file, webcam, server camera or RTSP stream
- Perception from a single camera feed
- Research and demonstration prototype
- Not intended for use in real vehicles

Methodology / Working



1. Detection
YOLOv8 finds pedestrians, cyclists and vehicles

2. Tracking
ByteTrack gives each object a persistent ID

3. Depth
MiDaS depth map gives per-object distance

4. Lane analysis
Canny, Hough lines and polynomial fit

5. Segmentation
Lightweight road and sky overlay

6. Risk engine
TTC from closing speed and distance

TTC = distance ÷ closing speed → Critical / Warning / Caution / Safe (thresholds adjustable live)

Technologies Used

Programming Language

- Python (backend and AI pipeline)
- JavaScript, HTML and CSS (dashboard)

AI / Vision Models

- YOLOv8n (Ultralytics) for detection
- ByteTrack for tracking, MiDaS for depth
- OpenCV for lane detection

Framework / Software

- Flask with MJPEG stream and Server-Sent Events
- PyTorch
- Chart.js and Canvas

Hardware & Input Sources

- CUDA GPU or CPU (GPU recommended)
- Single camera: video file, webcam or RTSP stream
- Demonstrated on a CPU-only machine

Expected Outcome / Result

What the system achieves

- Real-time detection and tracking of road users
- Distance and TTC from a single camera
- Four-level collision risk and lane-departure alerts
- Interactive dashboard with event log and CSV export

Limitations

- Monocular distance is an estimate, not ground truth
- Lane detection may struggle at night, in rain or with faded markings
- CPU-only machines run slower; GPU recommended

Future Scope

- Deep-learning lane model (LaneNet, UFLD)
- Traffic-sign and traffic-light recognition
- Camera calibration for metric distances
- Radar or LiDAR fusion
- Edge deployment (Jetson, Raspberry Pi)

Conclusion SafePilot shows that a single camera, open-source AI models and a web dashboard are enough for a practical ADAS prototype. It is low-cost, interactive and extensible.

Expected Outcome / Result: Dashboard Screenshots

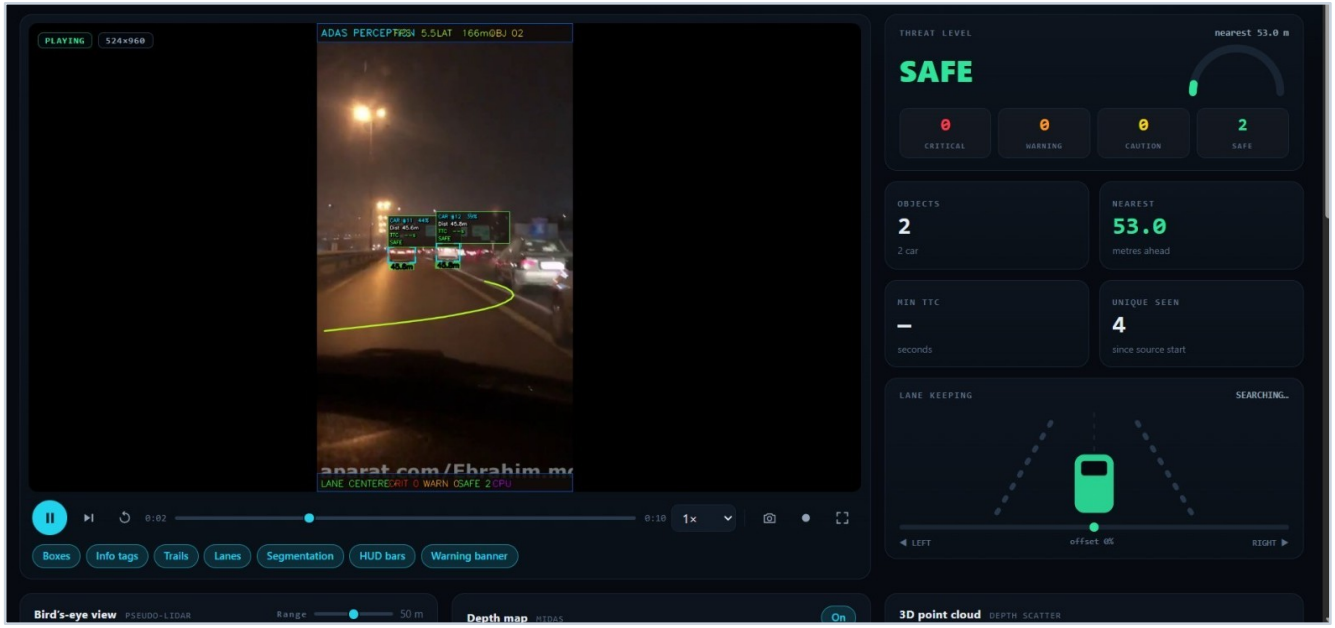


Fig. 1: Live detection, threat level, KPI cards and lane keeping

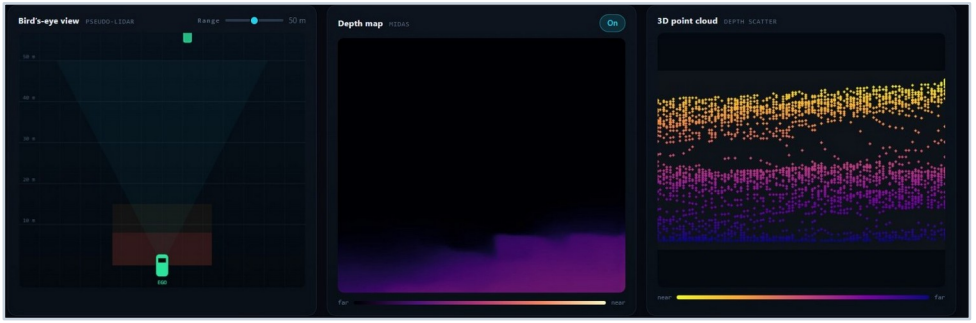


Fig. 2: Bird's-eye view, depth map and 3D point cloud

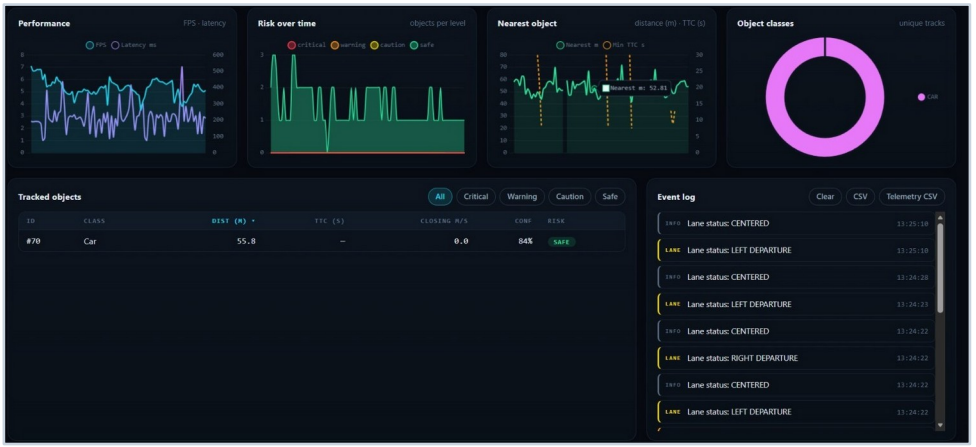


Fig. 3: Live charts, tracked objects and event log

Observed in a demo run on CPU: about 5 FPS, latency 100–320 ms (test video).

References

Research Papers

[1] Y. Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," European Conference on Computer Vision (ECCV), 2022. https://arxiv.org/abs/2110.06864

[2] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," arXiv:1907.01341. https://arxiv.org/abs/1907.01341

[3] J. Canny, "A Computational Approach to Edge Detection," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 8, no. 6, 1986. https://doi.org/10.1109/TPAMI.1986.4767851

Software and Documentation

[4] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," 2023. https://docs.ultralytics.com/models/yolov8

[5] OpenCV documentation. https://docs.opencv.org

[6] Flask documentation. https://flask.palletsprojects.com

[7] PyTorch. https://pytorch.org

[8] Chart.js. https://www.chartjs.org

[9] Project repository. https://github.com/uday6725

Thank You | Questions?